CrowdSecurity Auth – Full Write-up

1. Challenge Overview

The **CrowdSecurity Auth** is an ELF 64-bit binary designed to test static analysis and algebraic reversal skills. The program asks for a username and a password; it then validates the password against a specific username using a custom arithmetic algorithm.

- **Target Username:** CrowdSecurity
- **Goal:** Reverse-engineer the validation function to generate the correct password.

---

2. Reconnaissance & Initial Triage

Static Analysis with Radare2

We begin by analyzing the binary to identify key functions and embedded strings.

```
# Open the binary and perform full analysis
r2 -A auth

# List all strings in the data sections
[0x00001100]> iz
```

Index	V-Address	Length	String
0	0x00002008	24	give me your user name:
1	0x00002024	23	give me your password:
2	0x00002040	41	Good Job\nGo ahead and submit the password
3	0x0000206a	15	Access denied!!

Identifying the Entry Point

The string references point us toward the main logic. Searching for the main function:

```
[0x00001100]> afl ~main
0x0000130b    4    178 main
```

---

3. High-Level Code Analysis (Decompilation)

Using Ghidra, we can extract the C-style logic from the assembly. The program is divided into two primary functions.

A. The Main Wrapper (FUN_0010130b)

This function manages the user interface and memory. It allocates **50 bytes (0x32)** for inputs and calls the validator.

```c
undefined8 FUN_0010130b(void) {
  int iVar1;
  void *pvVar2; // Username Buffer
  void *pvVar3; // Password Buffer

  pvVar2 = malloc(0x32);
  pvVar3 = malloc(0x32);

  printf("give me your user name: ");
  __isoc99_scanf("%s", pvVar2);
  printf("give me your password: ");
  __isoc99_scanf("%s", pvVar3);

  // Call the verification function
  iVar1 = FUN_001011e9(pvVar2, pvVar3);

  if (iVar1 == 0) {
    puts("Access denied!!");
  } else {
    puts("Good Job\nGo ahead and submit the password");
  }
  return 0;
}
```

B. The Verification Logic (FUN_001011e9)

This function contains a hardcoded array of 13 integers (local_58) and a loop that performs character-by-character math.

```
undefined8 FUN_001011e9(char *username, long password) {
  int i;
  int local_58 [14]; // Target Table

  // Initialization of the verification table
  local_58[0] = 0x74;   local_58[1] = 0x143;  local_58[2] = 0x21b;
  local_58[3] = 0x334;  local_58[4] = 0x3a1;  local_58[5] = 0x456;
  local_58[6] = 0x50b;  local_58[7] = 0x63c;  local_58[8] = 0x719;
  local_58[9] = 0x889;  local_58[10] = 0x915; local_58[0xb] = 0x76b;
  local_58[0xc] = 0x85c;

  i = 0;
  do {
    if (strlen(username) <= (ulong)i) return 1; // Success if all chars pass

    // The Algorithm
    int u = (int)username[i];
    int p = (int)*(char *)(password + i);
    int L = local_58[i];

    if (((u + p) * (i + 1) - u) + i != L) {
      return 0; // Fail immediately if math is wrong
    }
    i++;
  } while(true);
}
```

---

4. Algorithmic Reversal

The core of the challenge is the expression:

To solve for the password (**p**) given the username (**u**), index (**i**), and target value (**L**), we can rearrange the equation step-by-step using algebraic isolation.

The Original Equation

((u + p) × (i + 1) - u) + i = L

Step-by-Step Derivation

1. Remove the trailing index (i):
   Subtract i from both sides of the equation.
   (u + p) × (i + 1) - u = L - i
2. Move the constant u to the right:
   Add u to both sides to further isolate the term containing p.
   (u + p) × (i + 1) = L - i + u
3. Divide by the multiplier (i + 1):
   Divide both sides by the (i + 1) term.
   u + p = (L - i + u) / (i + 1)
4. Isolate the password (p):
   Subtract u from both sides to find the final formula.
   p = [(L - i + u) / (i + 1)] - u

Final Formula

The password can be calculated using:

p = ((L - i + u) / (i + 1)) - u

---

5. Automated Solver (Python)

Using the derived formula, we can write a script to compute all 13 characters of the password instantly.

```python
#!/usr/bin/env python3
# CrowdSecurity Crackme Solver

USERNAME = "CrowdSecurity"
# Extracted from FUN_001011e9
TARGET_TABLE = [
    0x74, 0x143, 0x21b, 0x334, 0x3a1,
    0x456, 0x50b, 0x63c, 0x719, 0x889,
    0x915, 0x76b, 0x85c
]

def solve():
    recovered_password = ""

    print(f"{'Index':<6} | {'User Char':<10} | {'Table Val':<10} | {'Pass Char'}")
    print("-" * 50)

    for i in range(len(TARGET_TABLE)):
        u = ord(USERNAME[i])
        L = TARGET_TABLE[i]

        # Apply the inverse math: p = ((L - i + u) / (i + 1)) - u
        p_ascii = ((L - i + u) // (i + 1)) - u
        char = chr(p_ascii)
        recovered_password += char

        print(f"{i:<6} | {USERNAME[i]:<10} | {hex(L):<10} | {char}")

    return recovered_password

if __name__ == "__main__":
    pw = solve()
    print("-" * 50)
    print(f"[+] Final Credentials >> {USERNAME}:{pw}")
```

---

6. Final Results

Executing the solver yields the following character mapping:

Index	Username Char	Table (L)	Password Char
0	C	0x74	**t**
1	r	0x143	**h**
...	...	...	...
12	y	0x85c	**4**

Correct Credentials:

- **Username:** CrowdSecurity
- **Password:** thisisapass34

Conclusion

This challenge highlights a common weakness in custom verification routines: **lack of inter-dependence**. Because each character is checked independently of the others, the entire password can be cracked by solving N simple linear equations rather than one complex cryptographic problem.

8

fun_00101169 → (char* user, long password)

local_20 = (long) in_fs_offset + 0x28) → حماية للاستاك

local_58[0] = | المعفوظة دى

[1] ← لها اول التواتيج Dec ← اكبر من 255

غدى مش حروف

I = 0

if [username[I] + Password[I]]

*(I+1) - username[i] + i ≠ local58[I] → L

الشرط الاساسى :-

[I = I + 1]

$(U+P)\times(i+1) - U + i = L$

$\frac{L - i + U}{i+1} - U = P$

A-Z → 65 - 90

a-Z → 97 - 122

0-9 → 48 - 57

char ~~character~~ '0'

char = '0'

deci = ord (char)

Print(deci

deci = 65

ch = chr(deci)

Print(ch)

Cal ← C 0 → 67 Ascii

r 1 → 114

o 2 → 111

w 3 119

d 4 100

Cal ← S 5 83

e 6 101

c 7 99

u 8 117

r 9 114

i 10 101

t 11 116

y 12 121

Local-58 = L

74h = 116

143 h = 323

21b = 539

334 h = 820

391 h = 929

456 h = 1110

50bh = 1291

63Ch = 1596

719h = 1817

889h = 2185

915h = 2325

~~[illegible]~~

76bh = 1899

85Ch = 2140

i = 0

$\frac{116 - 0 + \cancel{67}}{0+1} - \cancel{67} = 116 = t$

i = 1 - - - -

Python script